Eight capabilities

What tinyCode does, as eight promises. Each page maps the mechanisms inside one capability by status: what stays, what evolves, what is new, and what is built but off until measured. The ids (A1, C12, E17, ...) are the feature ids used in the release notes. 46 mechanisms in all: 26 stay, 3 evolve, 6 new, 11 options.

Stays on, unchanged · **Evolves** kept, change in the backlog · **New** candidate, off until measured · **Option** built, off until measured

1

Every call fits the window

A long task never fills a 16k window. Code builds each call's context; old tool output is masked, not kept.



2

Fast on your machine

The server reuses what it has already read, so long runs don't slow down with every call.



3

Takes the shortest path

Say what you want in plain words; tinyCode picks the workflow and tries the cheapest fix first.



4

Edits that land

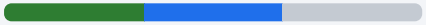
The model's edits apply and keep the code tidy.



5

Never loses your best work

You end with the best version tinyCode reached, not the last one.



6

Checks by running, not by believing

"done" means the project's checks pass, run by code, in Python and in TypeScript/JavaScript.



7

Gets unstuck

When a run stops improving, tinyCode changes course instead of repeating itself.



8

From idea to app

Research a question, plan an app, and build it task by task, each in a small context.



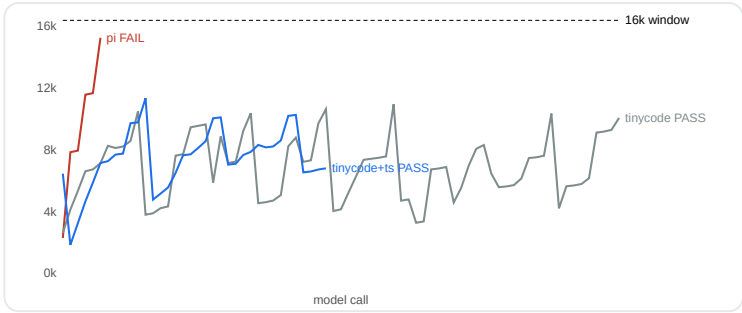
CAPABILITY 1 OF 8

1 Every call fits the window

A long task never fills a 16k window. Code builds each call's context; old tool output is masked, not kept.

EVIDENCE

Plain Pi ran out of window 8 times in two rounds (2026-11-10, 2026-11-11); tinyCode never did. Its highest call was 15.1k (connect, right after tinyCode stepped back and dropped its masks). E28, built 2026-11-11 behind `limits.stepBackKeepsMasks`, keeps those masks; a replay takes that call to 10.2k.



Context of every call, bowling, 16k: plain Pi (red) fills the window on call 6; tinyCode never passes 11.3k.

Stays · 6

on, unchanged

- Short tool results and file views (~1.5k tokens, ~100 lines) — C1 C2
- Old tool output masked; harder under pressure — C3 C9
- Finished work folded into notes; checkpoints restart from a summary — C7 C8 C10 C11
- Oversized AGENTS.md / CLAUDE.md replaced by a digest — D1
- Side questions answered in a separate call; only the answer comes back — D3 D4
- A short text protocol in tinyCode's own calls; a todo list as working memory — D2 C18

Evolves · 0

kept, change in the backlog

none

New · 2

candidate, off until measured

- Long bash commands and writes refused once, with a way past (bash output is already capped by C1) — E23
- Masks that go with the run: the plan once /build runs, smaller whole fresh files, older copies of a re-read file — E27

Option · 3

built, off until measured

- A step-back keeps the masks already made (parked 2026-11-12: step-backs are now rare, 3 in 30 runs) — E28
- Terse system prompt; no repeated task in the first turn — E4 E13
- Older turns summarised (later; masking measured better) — E8

2 Fast on your machine

The server reuses what it has already read, so long runs don't slow down with every call.

EVIDENCE

Keeping tinyCode's per-turn note out of the cached prefix, and declaring the tool list once, raised reuse on recorded runs (2026-11-04).

Stays · 3
on, unchanged

- Prompt and tool list kept stable so the cache holds — `D5` `D6`
- Learns per model where to start — `B8` `A4`
- Chat-template fix for strict models (Mistral family) — `C20`

Evolves · 0
kept, change in the backlog

none

New · 0
candidate, off until measured

none

Option · 1
built, off until measured

- Fewer cache breaks from checkpoints and masks — `E12`

3 Takes the shortest path

Say what you want in plain words; tinyCode picks the workflow and tries the cheapest fix first.

EVIDENCE

Easy tasks finish in 2–3 calls with the project's own tests (TS gate, 2026-11-10).

Stays · 4
on, unchanged

- Workflows (fix, test, ...) are tools the model calls — A5
- Only fix and test are taken over by default — A3
- One quick fix try built by code, then a second; no pick call with one candidate file — B1 B2 B6 B7
- A failing test written first when a fix names none — B9

Evolves · 1
kept, change in the backlog

- Plain-words router (natural wording routes 5 of 18 for research and plan) — A1

New · 0
candidate, off until measured

none

Option · 1
built, off until measured

- A small-model router for unsure cases — A2

4 Edits that land

The model's edits apply and keep the code tidy.

EVIDENCE

The rewrite nudge solved more of the tasks where it fired (4 of 5, 2026-11-04).

Stays · 4
on, unchanged

Edit size limit; edits off by one indent still applied; a mistyped path gets the right one

`C6` `C16` `C17`

An edit block with a repeated marker line (`=======`, `<<<<<<< SEARCH`, `>>>>>>> REPLACE`) is read once, not written into the file (2026-11-11/12; in the recorded replies, 8 of 254 doubled the `=======`, 12 of 322 a SEARCH or REPLACE line)

`fix`

A whole-file rewrite gets a one-line nudge to edit instead

`C21`

The project's formatter runs on changed files

`B10`

Evolves · 0
kept, change in the backlog

none

New · 0
candidate, off until measured

none

Option · 1
built, off until measured

After a rewrite, says what was lost and how to get it back

`E1`

5 Never loses your best work

You end with the best version tinyCode reached, not the last one.

Stays · 1
on, unchanged

Best tree kept, offered back, and put back when a run gets worse or ends below it

B5 C12 C13

Evolves · 0
kept, change in the backlog

none

New · 1
candidate, off until measured

Also offer going back to the start when the best version has the wrong design

E22

Option · 1
built, off until measured

Put the best back on a kill or time-out (partly built)

E3

6 Checks by running, not by believing

"done" means the project's checks pass, run by code, in Python and in TypeScript/JavaScript.

EVIDENCE

TypeScript support passed its gate and is on by default (2026-11-10).

GAP

On build tasks the model's own tests passed while the hidden tests did not (2026-11-11). The acceptance check meant to close it failed its first step: the checks the model wrote failed on correct code in 29 of 29 files.

Stays · 2

on, unchanged

Runs the project's own test command; reads pytest, jest, vitest and node:test

`D8 (with D7` `D9)`

A fix is "solved" only when a failing check turns green; a check that already passes stops with the reason (2026-11-12; before, one task was reported fixed with nothing changed)

`fix`

Evolves · 0

kept, change in the backlog

none

New · 1

candidate, off until measured

Time limits on tests the model runs itself

`E24`

Option · 1

built, off until measured

The whole project checked after each build step (`stages.wholeCheck`)

`E25`

7 Gets unstuck

When a run stops improving, tinyCode changes course instead of repeating itself.

Stays · 3
on, unchanged

- Hands a failed quick try to Pi's loop with the best tree and a diagnosis — `B4`
- Repeated calls blocked; step back after repeated failures; one nudge when it stops early — `C4` `C5` `C15`
- A heartbeat line every minute, a notice when stuck — `C19`

Evolves · 1
kept, change in the backlog

- Stuck rule (3 test runs without progress): too early for late breakthroughs; gets "start over" as a second exit — `C14 → E22`

New · 0
candidate, off until measured

none

Option · 2
built, off until measured

- A fresh-context review after a failure, or when the loop stalls — `B3` `E2`
- A time limit sized for a delegating agent — `E10`

8 From idea to app

Research a question, plan an app, and build it task by task, each in a small context.

EVIDENCE

Off by default. Turned on, it first scored 4 of 9 against 6 of 9 without it (2026-11-11 morning). Since then the plan review works: on the plan task where plans kept missing a file it went from 0 of 5 to 4 of 5 overnight. /build finished its first task after a parser fix, then stopped at task 2. Two causes are fixed since (the task's files reach the ladder; a partial build no longer says "stop"). /build is not yet re-measured.

Stays · 3

on, unchanged

Plans read the files their tasks name before they are final (`stages.planReview`, **on by default since 2026-11-12**: typed /plan 12/15 against 9/15, no task worse; it may name a file the request protects). Checks are printed in plans while E21b is off

`E21a`

What a build step carries: investigated; the largest peaks follow a step-back (→ E27, E28)

`E26`

Recorded web answers for repeatable benches

`E16`

Evolves · 1

kept, change in the backlog

/research, /plan, /build: off by default until the changes below are measured

`A3`

New · 2

candidate, off until measured

Split into plan and build only when the task won't fit in one context (deferred: needs evidence)

`E18`

Research looks locally first, and drops a source that fails (measured: finds and cites local pinned code 4 of 6 against 0; a cost bug fixed, re-measure pending)

`E19`

Option · 1

built, off until measured

/build runs each task's own check from the plan (`stages.buildChecks`; not measured). Checks are printed in plans while this is off

`E21b`

OUT OF THE LIST

Retired

Removed, proposed for removal, or closed as done.

An acceptance check written from the request (denied and removed 2026-11-12: the written checks failed on correct code in 29 of 29 files; its runner stays, for E21b's task checks)

`E17`

Research facts carry quotes checked by code (dropped 2026-11-12: did not help the task it was built for)

`E20`

Lifting Pi's 4096 clamp; a smaller tool set below 12k (with the 8k target)

`E14` `E15`

Compact data formats for tool results (proposed: tool results are file text, not records)

`E5` `E6`

A review after every edit (proposed: dropped in its design)

`E9`

Closed as done or not a feature: short report template (rest in E20), tokenizer check

`E7` `E11`